Documento de diseño y funcionalidades principales

Proyecto AIVA_2026

Referencia: DIS-2026-001

Versión 1.0

Equipo desarrollador:
Jorge Lozoya Astudillo
Félix Martínez Alonso

Preparado para:
José Francisco Vélez Serrano

12 de abril de 2026

Índice

1. Introducción **2**
1.1. Descripción de las técnicas de visión artificial 2
1.2. Validación y pruebas del sistema 2
1.3. Repositorio del proyecto 4

2. Diseño de la Solución **4**
2.1. Diagrama de Clases 4
2.2. Explicación del diagrama de clases 4
2.2.1. Detector 4
2.2.2. DetectionResult 5
2.2.3. SolderDefect 5
2.2.4. Camera 5
2.2.5. PCBImage 5
2.3. Diagrama de secuencia del entorno principal 6
2.4. Explicación del diagrama de secuencia del entorno principal 6
2.5. Diagrama de secuencia del motor de inferencia 6
2.6. Explicación del diagrama de secuencia de inferencia 8
2.7. Diagrama de actividad del sistema general 8
2.8. Explicación del diagrama de actividad general 8
2.9. Diagrama de actividad del algoritmo de visión 8
2.10. Explicación de la lógica del algoritmo 10

1. Introducción

El presente desarrollo surge como respuesta a la necesidad de automatizar el control de calidad en la fabricación de placas de circuito impreso (PCB). El objetivo principal es la implementación de un sistema de visión artificial capaz de identificar defectos estructurales en las soldaduras de forma autónoma, integrándose en un flujo de trabajo industrial que combina hardware de captura (Raspberry Pi), procesamiento mediante una red neuronal y control robótico. Mediante esta solución, se busca reducir el margen de error humano y optimizar los tiempos de ciclo en la línea de montaje.

1.1. Descripción de las técnicas de visión artificial

La solución técnica se basa en la arquitectura de red neuronal **YOLO (You Only Look Once)**, específicamente la versión implementada por la librería *Ultralytics*. Esta técnica de detección de objetos en una sola etapa (*one-stage detector*) permite localizar y clasificar anomalías en tiempo real con una alta precisión, extrayendo las coordenadas espaciales y el nivel de confianza de cada predicción directamente de la imagen de entrada.

Como referencia técnica y bibliográfica para el motor de inferencia utilizado, se destaca el trabajo de:

> Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Version 8.0.0). Software disponible en https://github.com/ultralytics/ultralytics.

1.2. Validación y pruebas del sistema

Para garantizar la robustez del software, se ha desarrollado una batería de pruebas unitarias automatizadas utilizando la librería `unittest`. A continuación, se describen tres de los casos de prueba más representativos que validan el comportamiento del sistema ante diferentes escenarios.

- **Ejemplo 1: Tolerancia a fallos de hardware (`test_capture_device_fails_gracefully`).** Este test verifica la robustez física del sistema. Simula una pérdida de conexión con la cámara de la Raspberry Pi intentando acceder a un puerto inexistente. El test pasa exitosamente al comprobar que el programa no sufre un cierre abrupto, sino que intercepta el fallo y lanza una excepción controlada (`ConnectionError`) notificando al operario.
- **Ejemplo 2: Inferencia de visión artificial (`test_inference_output`).** Valida el núcleo del motor de inteligencia artificial. Se inyecta una matriz de imagen vacía (completamente negra) directamente al modelo YOLO para simular una placa perfecta. El test verifica que la red neuronal es capaz de procesar la matriz matemáticamente sin errores de arquitectura y que, de forma lógica, devuelve una lista de defectos de longitud cero.
- **Ejemplo 3: Generación del reporte por lotes (`test_add_and_batch_xml`).** Este caso evalúa la capa de salida de datos. Se inyectan manualmente múltiples instancias de `SolderDefect` con coordenadas simuladas dentro de un contenedor `DetectionResult`. El test comprueba que la exportación final agrupa correctamente todos los elementos y que las etiquetas del string resultante (`<x>`, `<y>`, `<detection_count>`) tienen el formato XML exacto requerido por el brazo robótico.

Como evidencia de la cobertura total de las pruebas, en la Figura 1 se muestra la ejecución satisfactoria de los tests unitarios desarrollados.

Para asegurar la calidad del código de forma automatizada y profesional, se ha implementado un flujo de integración continua mediante GitHub Actions. Este entorno ejecuta automáticamente todos los tests ante cada nueva contribución al repositorio, garantizando que ninguna actualización comprometa las funcionalidades existentes. En la Figura 2 se observa el historial de ejecuciones exitosas en el servidor remoto.

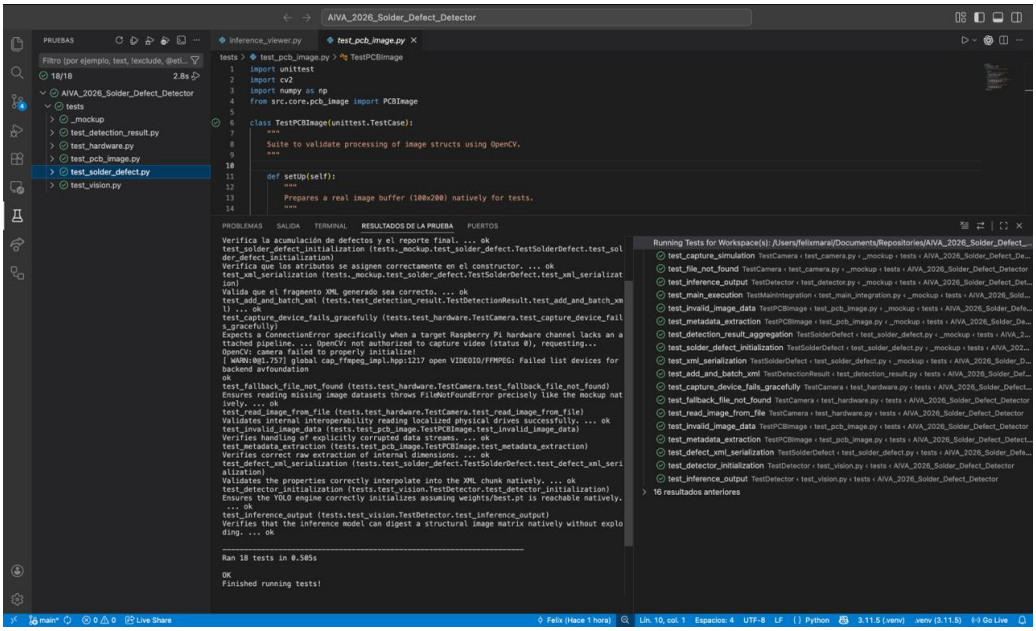


Figura 1: Resultados de la batería de pruebas unitarias en el entorno de desarrollo.

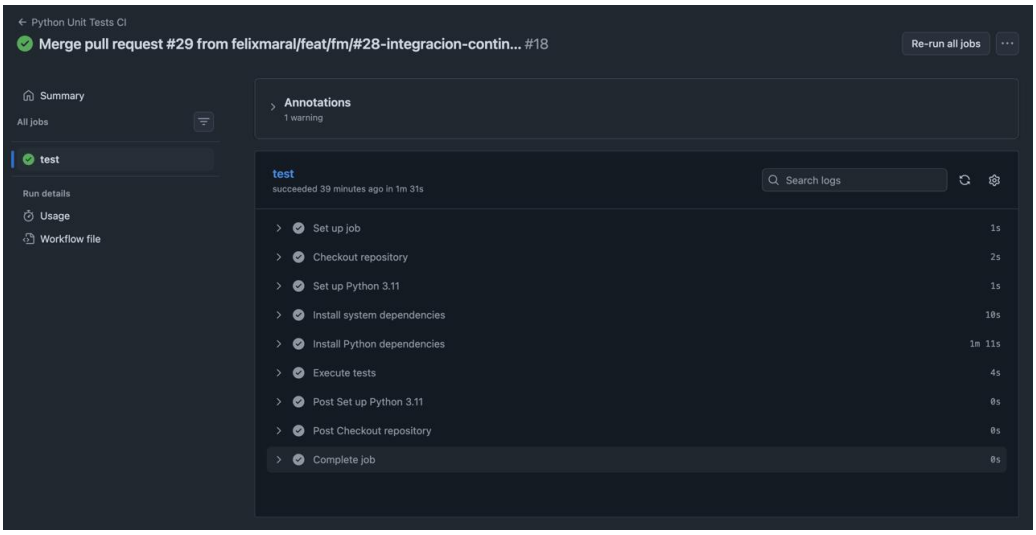


Figura 2: Interfaz de GitHub Actions mostrando el flujo de integración continua finalizado con éxito.

1.3. Repositorio del proyecto

Todo el código fuente de la biblioteca, el motor de inferencia y la documentación técnica asociada están disponibles para su consulta y revisión en el siguiente enlace de GitHub:

https://github.com/felixmaral/AIVA_2026_Solder_Defect_Detector

2. Diseño de la Solución

En esta sección se detalla la arquitectura del sistema de inspección de soldadura, desglosando las clases principales, sus interacciones y el flujo lógico del algoritmo de visión.

2.1. Diagrama de Clases

A continuación se presenta la estructura estática del sistema, donde se definen las responsabilidades de cada componente y las relaciones de dependencia entre ellos.

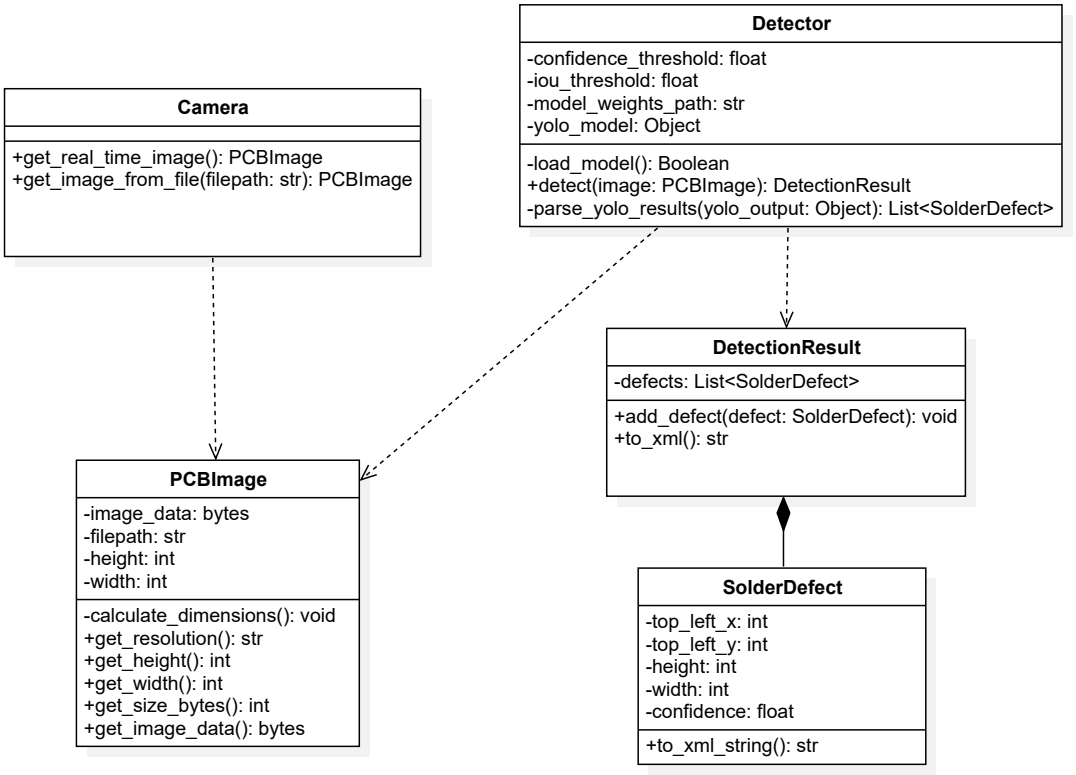


Figura 3: Diagrama de clases (Alta Resolución).

2.2. Explicación del diagrama de clases

El sistema se ha diseñado bajo los principios de la programación orientada a objetos, asegurando el encapsulamiento y la separación de responsabilidades. A continuación, se detalla la función de cada clase:

2.2.1. Detector

Esta clase es el núcleo de visión del sistema. Su función es aislar la librería de inteligencia artificial (YOLO) del resto del código, de modo que si en el futuro cambiamos de modelo o de

versión, solo haya que tocar esta parte. Internamente, guarda de forma privada la configuración clave, como los umbrales de confianza.

Su método principal es `detect()`, que se encarga de procesar la imagen. Lo más destacado de su diseño es el método privado `parse_yolo_results()`. Este actúa como un traductor: coge los números brutos que devuelve YOLO y los convierte en nuestros objetos `SolderDefect`. Esto hace que los datos sean fáciles de manejar por el resto de nuestro programa.

2.2.2. DetectionResult

Esta clase funciona como una caja donde agrupamos todos los fallos encontrados en una misma placa. Su objetivo es separar el análisis de la imagen de la generación de archivos, separando responsabilidades para facilitar el mantenimiento.

Guarda los fallos en una lista privada (`defects`) a la que solo se pueden añadir elementos usando el método público `add_defect()`. Una vez que el `Detector` ha terminado su trabajo, se llama al método `to_xml()`, que recorre esta lista y crea automáticamente el archivo final con el formato exacto que pide el brazo robótico.

2.2.3. SolderDefect

Esta clase representa un único fallo de soldadura detectado en la placa. Guarda la información que necesita el robot: dónde está ubicado exactamente (sus coordenadas y tamaño) y la seguridad que tiene la red neuronal sobre ese fallo (su porcentaje de confianza).

Para evitar fallos en tiempo de ejecución, todos sus datos se guardan como privados. Su única función hacia el exterior es el método público `to_xml_string()`, que se encarga de devolver un texto corto con sus coordenadas (la etiqueta `<defect>... </defect>`) listo para insertarse en el archivo XML principal.

2.2.4. Camera

Esta clase es nuestro puente con el hardware, la cámara acoplada a la Raspberry Pi. Sirve para estandarizar la forma en la que el sistema toma una fotografía, sin importar de dónde venga.

Como solo actúa como un servicio de paso, no necesita guardar ningún dato interno (no tiene atributos). Ofrece dos métodos públicos clave: `get_real_time_image()` para capturar en producción, y `get_image_from_file()` para cargar fotos de prueba durante el desarrollo. Ambos devuelven siempre un objeto `PCBImage`, asegurando que el resto del sistema trabaje sin preocuparse del origen de la imagen.

2.2.5. PCBImage

Esta clase actúa como el tipo de dato oficial para manejar las fotografías dentro del sistema. Su objetivo es encapsular los bytes crudos de la imagen y sus características asociadas (como la ruta o el tamaño) en una estructura unificada. Con esto, evitamos pasar variables sueltas entre funciones.

Para garantizar que la información de la imagen no se altere accidentalmente durante el proceso, sus atributos principales (`image_data`, `height`, `width`) son estrictamente privados. Si otra parte del programa, como el `Detector`, necesita conocer las medidas exactas, debe solicitarlo a través de sus métodos de consulta públicos. Asimismo, para permitir que motores de inteligencia artificial externos analicen la fotografía sin romper estas reglas de encapsulamiento, se ha incorporado el método `get_image_data()`, el cual actúa como la única pasarela segura para extraer los bytes crudos de la imagen. De esta forma, nos aseguramos de que el manejo de las imágenes sea siempre predecible y organizado.

2.3. Diagrama de secuencia del entorno principal

Este diagrama detalla la interacción entre los componentes del sistema, centrándose en el ciclo de vida de una petición de inspección desde el programa principal.

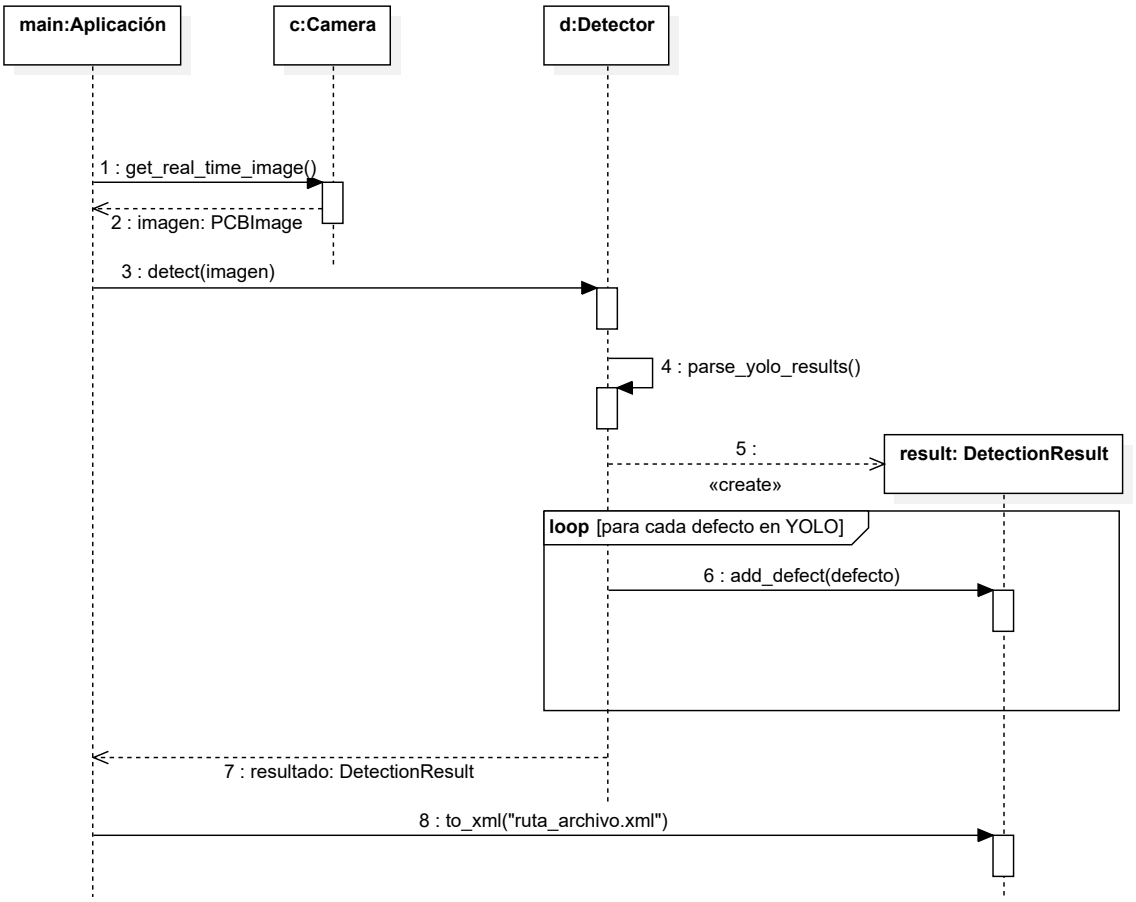


Figura 4: Interacción entre componentes durante el proceso de detección.

2.4. Explicación del diagrama de secuencia del entorno principal

Este diagrama ilustra el flujo de ejecución general del sistema desde la perspectiva del orquestador principal (`Main`). Muestra de forma cronológica cómo se coordinan las distintas clases para procesar una placa, desde que se captura la fotografía hasta que se exportan los datos en el formato requerido por el cliente.

El proceso comienza cuando el programa solicita una imagen a la instancia de `Camera`. Con la fotografía en memoria, el orquestador invoca el método `detect()` del motor lógico (`Detector`). Internamente, tras consultar al modelo de inteligencia artificial, el detector utiliza su método privado `_parse_yolo_results()` para traducir los datos matemáticos brutos.

Acto seguido, crea un contenedor `DetectionResult` y, mediante un bucle, va registrando cada fallo individual utilizando `add_defect()`. Una vez finalizada la detección, se devuelve el control al programa principal, el cual ordena la generación del archivo final llamando a `to_xml()`.

2.5. Diagrama de secuencia del motor de inferencia

Este diagrama profundiza en la lógica interna del proceso de detección, mostrando la interacción entre el motor lógico, la librería de inteligencia artificial y la instanciación de los fallos de soldadura.

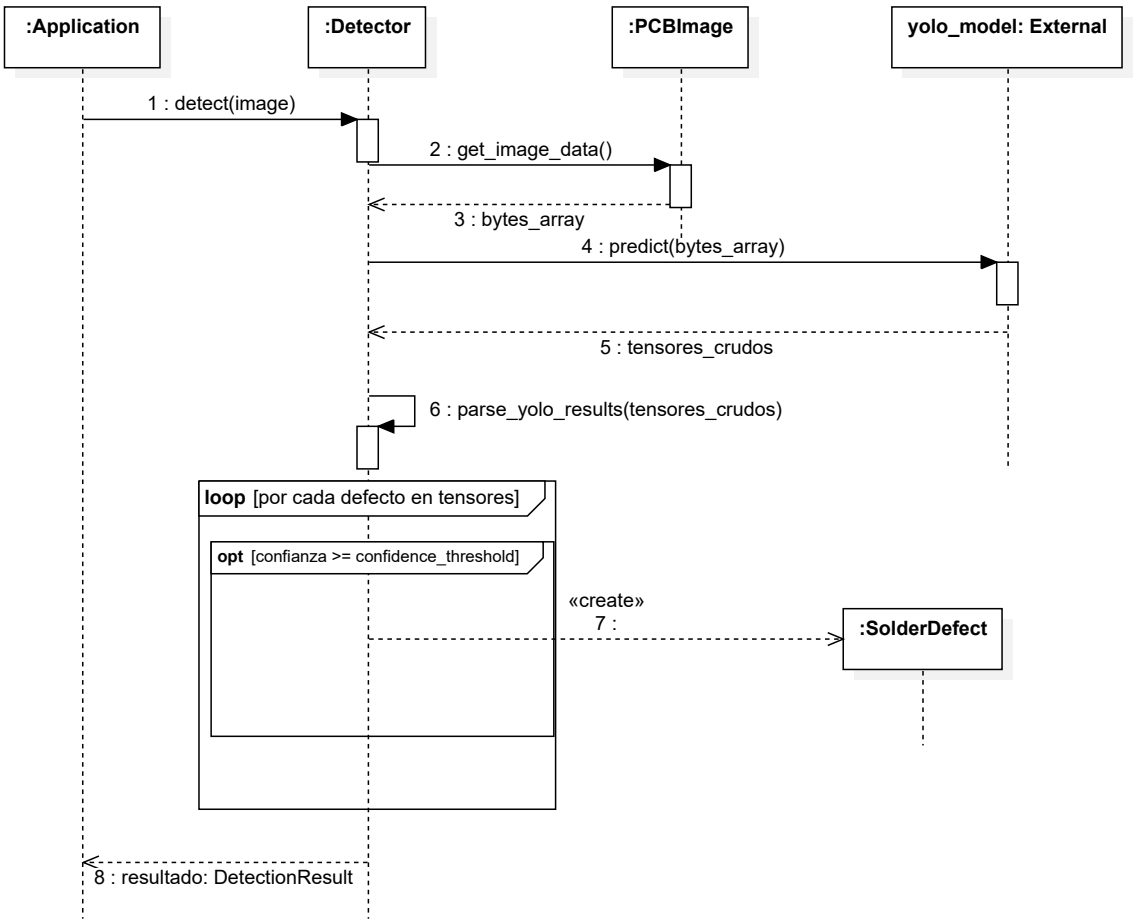


Figura 5: Detalle del proceso de inferencia y traducción de resultados.

2.6. Explicación del diagrama de secuencia de inferencia

Este segundo diagrama detalla el flujo de trabajo en el momento más crítico del sistema: la comunicación con el modelo de YOLO y la posterior traducción de sus resultados. Su propósito es ilustrar cómo nuestro diseño aísla la complejidad matemática de la red neuronal externa para convertir sus salidas en objetos útiles y estructurados para el brazo robótico.

El proceso se inicia cuando el `Detector` extrae los bytes crudos de la imagen y los envía a la librería externa. YOLO devuelve una serie de tensores que son interceptados por el método privado `parse_yolo_results()`. A partir de aquí, el sistema entra en un bucle para evaluar cada predicción detectada. Mediante un bloque condicional (`opt`), se aplica la regla de negocio principal: si la confianza de la predicción supera el umbral configurado (`confidence_threshold`), el sistema toma esos números y crea una nueva instancia del objeto `SolderDefect`. Si la confianza es baja, la predicción se descarta.

2.7. Diagrama de actividad del sistema general

Este diagrama representa el flujo operativo global.

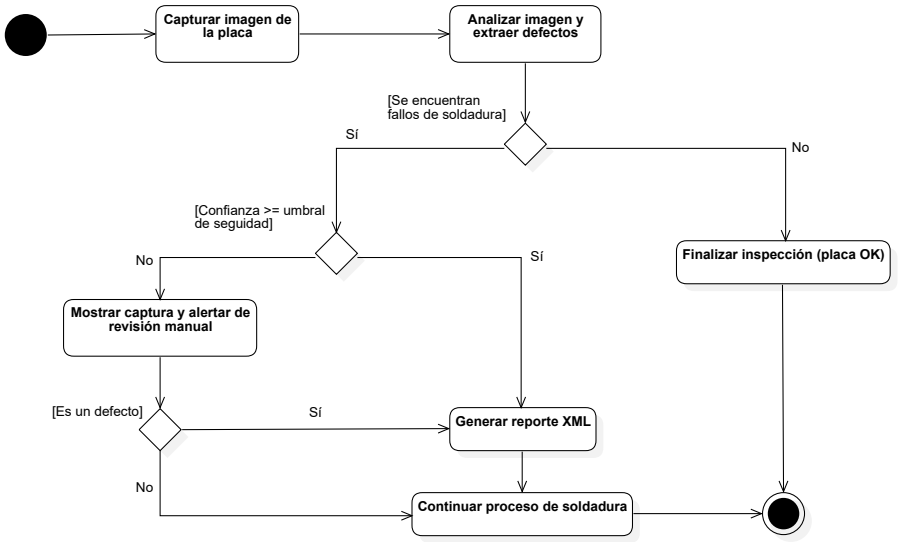


Figura 6: Flujo de actividad del proceso de inspección y validación.

2.8. Explicación del diagrama de actividad general

Este diagrama ilustra el ciclo de vida completo de la inspección de una placa. El proceso secuencial comienza con la captura y el análisis fotográfico; si la placa está limpia de errores, el ciclo finaliza inmediatamente. El punto clave radica en el doble filtrado cuando se detectan posibles fallos: el sistema evalúa primero si la predicción supera el umbral de seguridad, realizando la generación del reporte XML si la certeza es alta. Por el contrario, si la detección es dudosa, el flujo se desvía hacia un protocolo de revisión manual donde un operario humano tiene la última palabra para confirmar el defecto o descartarlo como un falso positivo. De esta forma, todos los caminos convergen en la continuación del proceso de soldadura, garantizando que el brazo robótico actúe de forma ininterrumpida.

2.9. Diagrama de actividad del algoritmo de visión

Este diagrama describe la lógica procedimental interna del motor de detección, detallando el manejo de datos desde la entrada de la imagen hasta la estructuración de la salida.

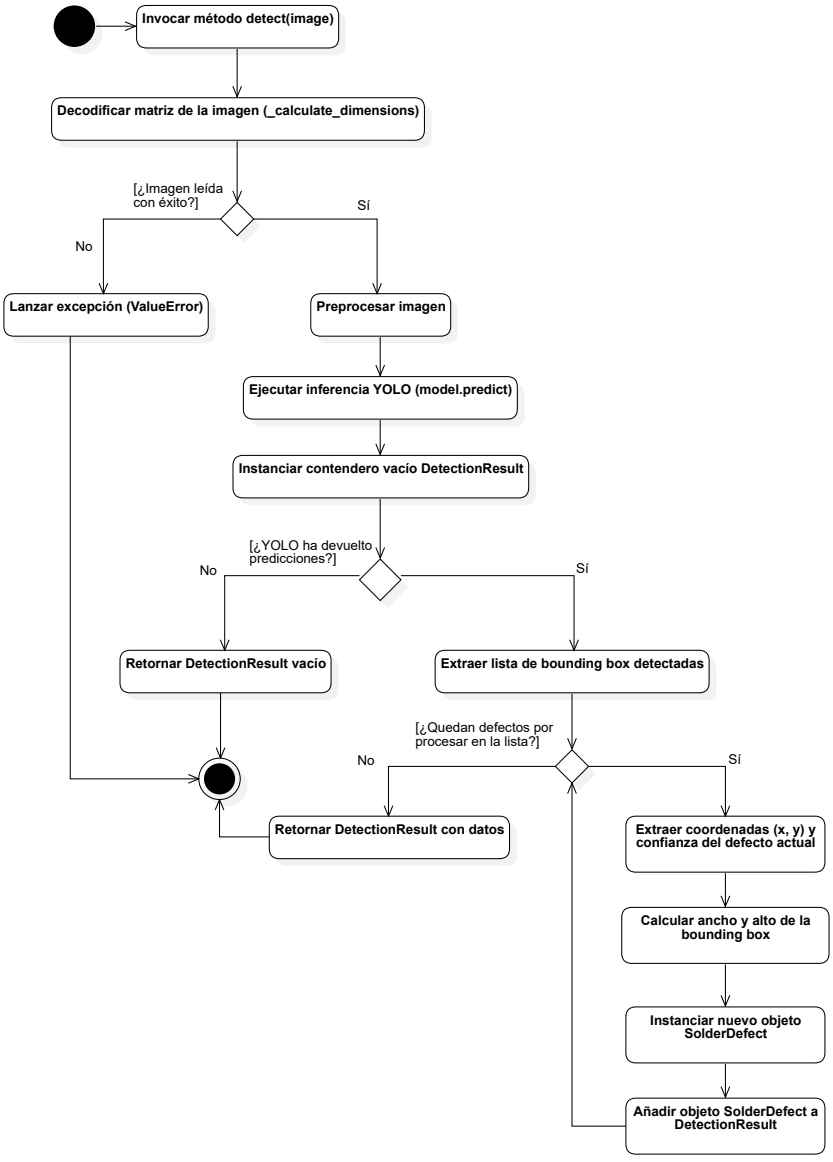


Figura 7: Lógica de ejecución del algoritmo de inferencia YOLO.

2.10. Explicación de la lógica del algoritmo

Este diagrama detalla el flujo de ejecución a nivel de código del algoritmo principal de inspección, mapeando el comportamiento del método `detect` de la clase `Detector`. El proceso comienza con una fase de validación y preprocesamiento, asegurando que la imagen de la placa decodificada posea el formato correcto (escala de grises en 3 canales) antes de alimentar a la red neuronal. Tras invocar la inferencia de YOLO, el flujo presenta un comportamiento robusto frente a ausencias de error: si la red no devuelve anomalías, el sistema aborta de forma temprana devolviendo un contenedor de resultados vacío. El núcleo del algoritmo reside en su fase final, donde un bucle iterativo actúa como patrón adaptador, extrayendo los tensores de cada predicción (coordenadas espaciales y porcentajes de confianza) para transformarlos, uno a uno, en instancias sólidas del objeto `SolderDefect`, encapsulándolas finalmente en el reporte global de la placa.